

Experiment 2: Selection Sort – Memory-Constrained Embedded Device

Aim

To implement the **Selection Sort** algorithm in Python with a **swap counter** to demonstrate that Selection Sort performs at most **$n - 1$ swaps**, making it suitable for memory-constrained embedded devices.

Problem Statement

A low-memory IoT sensor device collects temperature readings and must sort them with **minimal write/swap operations** because each write to memory is costly. Implement Selection Sort and count the number of swaps performed.

Algorithm

Step 1: Start.

Step 2: Read the number of elements.

Step 3: Read the array elements.

Step 4: Initialize swap_count = 0.

Step 5: For each position:

- Find the minimum element in the remaining array.
- If it is not already in the correct position, swap it.
- Increase the swap counter.

Step 6: Display the sorted array.

Step 7: Display the total number of swaps.

Step 8: Stop.

Source Code (Python)

```
main.py
1 # Selection Sort with Swap Counter
2
3 n = int(input("Enter number of elements: "))
4
5 arr = []
6
7 print("Enter the elements:")
8 for i in range(n):
9     arr.append(float(input()))
10
11 swap_count = 0
12
13 for i in range(n - 1):
14     min_index = i
15
16     for j in range(i + 1, n):
17         if arr[j] < arr[min_index]:
18             min_index = j
19
20     if min_index != i:
21         arr[i], arr[min_index] = arr[min_index], arr[i]
22         swap_count += 1
23
24 print("Sorted array:", arr)
25 print("Number of swaps:", swap_count)
```

Output

```
Enter number of elements: 5
Enter the elements:
23.5
19.2
25.1
18.8
21.4
Sorted array: [18.8, 19.2, 21.4, 23.5, 25.1]
Number of swaps: 2

=== Code Execution Successful ===
```

Complexity Analysis

Aspect	Complexity
---------------	-------------------

Time Complexity	$O(n^2)$
------------------------	----------------------------

Space Complexity	$O(1)$
-------------------------	--------------------------

Maximum Swaps	$n - 1$
----------------------	---------------------------

Result

The Selection Sort algorithm was successfully implemented with a swap counter in Python. The program sorted the given elements correctly and displayed the total number of swaps performed, demonstrating that Selection Sort performs at most $n - 1$ swaps, making it suitable for memory-constrained embedded devices.